

Conv_Simple — Prog3-Modbus Polling Sequencer

Build the 4-rung VFD polling ladder, fix the v5.1 Channel + status-register bugs

Audience: First-time CCW ladder author who already programmed the Micro 820 in ST | **PLC:** Allen-Bradley Micro 820 (2080-LC20-20QBB)
Drive: AutomationDirect GS10 DURApulse, Modbus slave node 1
Project: MIRA_v5 | **IDE:** Connected Components Workbench (CCW) v22+
Primary reference: GS10 DURApulse User Manual + plc/GS10_Integration_Guide.md
This doc supersedes: plc/LD_BUILD_GUIDE_v5.md (the v5.1 4-rung build steps)
You will understand after reading: why the polling sequencer is a state machine driven by vfd_poll_step , what each MSG_MODBUS instance is actually reading/writing, and which v5.1 values were wrong.

❶ One-page cheat sheet (the values that absolutely must be right)		
What	Value	Source
PLC serial channel for embedded RS-485	2 (NOT 0 — the v5.1 doc was wrong)	Micro 820 ref manual; matches GS10_Integration_Guide.md §5
Drive RJ45 pin for D+ (SG+)	Pin 5 (green/white per T568B)	GS10 manual p.10; GS10_Integration_Guide.md §3
Drive RJ45 pin for D- (SG-)	Pin 4 (blue)	GS10 manual p.10; GS10_Integration_Guide.md §3
VFD slave address (Modbus Node)	1 (matches P09.00 on the keypad)	GS10 manual p.6; GS10_Integration_Guide.md §1
Baud / format	9600 8N2 RTU (P09.01=96, P09.04=13)	GS10 manual p.6
Command register (FC06 write)	0x2000 (dec 8192)	GS10 manual p.6; GS10_Integration_Guide.md §2
Frequency setpoint register (FC06 write)	0x2001 (dec 8193)	GS10 manual p.6; GS10_Integration_Guide.md §2
Fault reset register (FC06 write)	0x2002 (dec 8194), data = 2	GS10 manual p.6; GS10_Integration_Guide.md §6
FC03 monitor block read	Start 0x2103 (dec 8451), ElementCnt = 4 (covers 0x2103–0x2106 contiguous; do NOT read 6 — 0x2108/0x2109 are undocumented)	GS10 manual p.5; GS10_Integration_Guide.md §2 register map
Operation Status WORD register (FC03 read)	0x2101 (dec 8449) — different from Output Freq!	GS10 manual p.5; GS10_Integration_Guide.md §2
Cmd word: FWD + RUN (Mike's deployed value)	18 (0x0012)	GS10_Integration_Guide.md §2 line 102; bit math contradicts — see §4 below

What	Value	Source
Cmd word: REV + RUN (Mike's deployed value)	20 (0x0014)	GS10_Integration_Guide.md §2 line 104; bit math contradicts — see §4 below
Cmd word: STOP	1 (0x0001)	GS10 manual p.6

⚠ 3 bugs in plc/LD_BUILD_GUIDE_v5.md (v5.1) that this doc fixes

1. **Channel is wrong:** v5.1 says "Channel = 0 (built-in RS-485)" in every rung's config table. The Micro 820's *embedded* serial port is `Channel := 2`; channel 0 is reserved for a plug-in serial card that the 2080-LC20-20QBB does not have. GS10_Integration_Guide.md §5 already shows `Channel := 2` in the ST code — the build guide drifted from the integration guide and was never reconciled. **Fix:** set `read_local_cfg.Channel := 2`, `write_cmd_local_cfg.Channel := 2`, `write_freq_local_cfg.Channel := 2`, `fault_reset_local_cfg.Channel := 2` in Prog2 ST. Do NOT change the LD rungs themselves — Channel lives in the config struct that Prog2 ST loads each scan.
2. **Rung 1 is reading Output Frequency, not the status word — but the instance is named mb_read_status :** v5.1 sets `Target Addr = 8451 (0x2103)` and reads 6 registers. Per the GS10 register map, `0x2103` is **Output Frequency**, and the 6-register block `0x2103–0x2108` is the live *monitor* data (frequency, current, DC bus, output voltage, multi-step, power factor) — useful, but **not the operation status word**. The actual status bits (running / fault / direction / at-speed) live at `0x2101` (dec 8449). **Fix:** either rename the instance to `mb_read_monitor` (recommended — what the v5.1 code is actually reading) and add a second FC03 read at `0x2101` for the status word; or, if you only care about the monitor block, leave it but stop calling it "status".
3. **ElementCnt mismatch — v5.1 contradicts itself, and the safer value is 4:** Rung 1's header comment says "reads 4 registers" but the inline config table says `ElementCnt = 6`. The integration guide ST code (GS10_Integration_Guide.md §5) uses `ElementCnt := 4`. Per the GS10 register map, the contiguous documented block is `0x2103–0x2106` (Output Frequency, Output Current, DC Bus Voltage, Output Voltage) — exactly 4 registers. Reading 6 would extend into `0x2107` (Multi-Step Speed Sel — valid but rarely meaningful at runtime) and then `0x2108` (undocumented in the GS10 manual register map at §2). Reading undocumented registers is unsupported behavior — the drive may return garbage or a Modbus exception. **Fix:** use `ElementCnt := 4` in Prog2 ST for the monitor read. `read_data(1)` = Output Freq, `read_data(2)` = Output Current, `read_data(3)` = DC Bus, `read_data(4)` = Output Voltage. If you genuinely need Multi-Step Speed or Power Factor later, add a separate FC03 read for them; don't extend this one.

1 The Big Picture

You're going to make the PLC ask the VFD four questions, once every ~50 ms, forever. "What's your output frequency?" "Run forward at 30 Hz." "Frequency setpoint = 30.0 Hz." "Clear any active fault." Each question is one Modbus packet on the RS-485 wire. Each packet is fired by one ladder rung. A counter (`vfd_poll_step`) decides whose turn it is to fire — only one rung sends at a time, because you can only have one packet on the wire at a time without collisions.

2 The Real-World Problem

A conveyor sits on the floor with a 2-HP motor wired through a GS10 VFD. The motor's run/stop/direction is controlled from the PLC — not the VFD's keypad. The PLC needs to (a) tell the VFD what to do (run forward at 30 Hz), (b) read back what the VFD is actually doing (motor's at 28.4 Hz, drawing 1.7 A, no fault), and (c) clear faults when the operator presses the reset button. All of this happens over one twisted pair of wires using the Modbus RTU protocol.

The bench rig is one PLC, one drive, one motor, one ~6-foot RS-485 cable. No production gear. You can break things and restart them. That's the point.

3 The Simple Version First

Before the 4-rung sequencer exists, the simplest thing that works is **one** rung firing **one** `MSG_MODBUS` read continuously:

```
|
+---[ ]---always_true----[MSG_MODBUS]---mb_read_one-----+
|                                                             |
```

One read, every scan. The PLC fires another Modbus packet before the previous one has finished. The drive ignores you. The serial driver buffers up requests and eventually goes "out of sync". You see CE10 (comm timeout) on the VFD keypad and ErrorID = 55 in the live monitor. The simple version reads one register fine for about 200 ms, then everything jams.

Why the simple version works at all

`MSG_MODBUS`'s `IN` pin is rising-edge triggered. Even with `always_true` always high, the block only fires once per scan and waits for `.Done` before re-triggering. So you get one packet every few scans, which "works" for one register. The problem appears the moment you want to read AND write — two messages racing each other.

4 Why This Becomes a Problem

The conveyor needs four conversations: read monitor data, write the run command, write the frequency setpoint, clear faults. If you fire all four from rungs that are all enabled at the same time, the serial driver tries to send four packets stacked end-to-end. The Micro 820's embedded serial driver doesn't queue packets — it expects you to wait for one to complete before starting the next. Stack them and you get partial sends, garbled CRCs, and the drive's CE10 timeout fault.

The pain made concrete

Imagine four people trying to ask the same waiter four different questions at the same time. The waiter can answer one at a time. If everyone shouts, the waiter writes nothing down. You need a queue — and a way to take turns.

5 The Better Pattern — a step-driven sequencer

One integer (`vfd_poll_step`) cycles $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$, advanced by a TON timer in Prog2. Each ladder rung in Prog3 watches the step and fires its `MSG_MODBUS` **only when the step matches its number**. So at any given instant exactly one MSG is active. The wire never sees overlap.

Why this matters: A polling sequencer is a rotating spotlight. Only the rung the spotlight is on can speak. When the spotlight moves on, that rung shuts up and the next rung takes its turn. The TON timer is the clock that moves the spotlight; `vfd_poll_step` is the spotlight's current position.

This pattern is also what lets you add a fifth conversation later (e.g., "read motor temperature") without touching any of the existing rungs — bump the modulo, add a Rung 5, done.

6 CCW Step-by-Step Walkthrough

Open `MIRA_v5.ccws1n` in CCW. The pre-build checklist below has to be true before any of these steps work — most of it lives in Prog2 (the ST program) and the global variable list. If you skipped one, the rung will compile but report ErrorID = 55 the moment you go online.

Pre-build checklist

- ☐ CCW project `MIRA_v5` exists; controller is **2080-LC20-20QBB**.
- ☐ **Embedded Serial Port** → **Properties**: Protocol = Modbus RTU, Mode = Master, Baud = 9600, Parity = None, Stop Bits = 2, Handshake = None. *[Micro 820 ref manual]*
- ☐ Variables imported from `VARIABLES_IMPORT.csv`. Includes `vfd_poll_step`, `vfd_poll_active`, the four config structs, the four data buffers, and the four `MSG_MODBUS` instances.
- ☐ Prog2 (ST) loaded with the latest `Prog2.stf`. Prog2 owns the timer that advances `vfd_poll_step` and loads the four config structs each scan.
- ☐ `MbSrvConf.xml` copied to `Controller\Controller\` folder.
- ☐ Program execution order: **Prog2 first, Prog3-Modbus second**. Set in Project Organizer → Programs → Properties.

Add the Prog3-Modbus LD program

1. ☐ Project Organizer → right-click **Programs** → **Add** → **New LD: Ladder Diagram**.
2. ☐ Rename it **Prog3-Modbus**. Confirm execution order again (Programs → Properties).
3. ☐ Double-click to open the ladder editor.

CCW "no match" panic — don't click away

When you drop a contact and type a variable name like `vfd_poll_step`, the dropdown may briefly show "no match" while it indexes the variable list. Press Enter anyway — if the variable was imported from CSV, CCW will accept it. Only worry about "no match" if it persists after a build attempt.

Rung 1 — Read VFD Monitor Block

```

|
+---[EQU]-----[XIC]-----[MSG_MODBUS]-----+
|  A = B      vfd_poll_active  mb_read_monitor
|  A = vfd_poll_step
|  B = 1
|

```

Reads 4 registers starting at 0x2103 (Output Freq, Output Current, DC Bus Voltage, Output Voltage). Note instance renamed from v5.1's misleading `mb_read_status`; `ElementCnt` corrected from 6 to 4 (see bug #3).

1. ☐ Drag **EQU** from Toolbox to the rung. Source A = `vfd_poll_step`, Source B = `1`.
2. ☐ Drag **XIC** (NO contact). Variable = `vfd_poll_active`.
3. ☐ Drag **MSG_MODBUS** from Toolbox (search "MSG"). Instance name = `mb_read_monitor` (not `mb_read_status` — see bug #2).
4. ☐ Click each pin to wire variables:
 - `LocalCfg` = `read_local_cfg`
 - `TargetCfg` = `read_target_cfg`
 - `LocalAddr` = `read_data`

Config values Prog2 ST loads each scan (these go into the structs — not the rung):

Field	Value	Why
<code>read_local_cfg.Channel</code>	2	Embedded RS-485 is channel 2 on Micro 820 <i>[Micro 820 ref manual]</i>
<code>read_local_cfg.Cmd</code>	3	FC03 = Read Holding Registers <i>[Modbus spec]</i>
<code>read_local_cfg.ElementCnt</code>	4	Four monitor registers — 0x2103–0x2106 (freq, current, DC bus, V_out). Reading 6 lands in undocumented 0x2108 — don't do it.
<code>read_target_cfg.Addr</code>	8451 (0x2103)	Start of the monitor block <i>[GS10 manual p.5]</i>
<code>read_target_cfg.Node</code>	1	VFD's Modbus slave address (P09.00) <i>[GS10 manual p.6]</i>

Rung 2 — Write VFD Command

```

|
+---[EQU]-----[XIC]-----[MSG_MODBUS]-----+
|  A = B      vfd_poll_active  mb_write_cmd
|

```

```
A = vfd_poll_step
B = 2
```

Writes one register to 0x2000 (Control Command). Value comes from `vfd_cmd_word`, loaded into `write_cmd_data(1)` by Prog2 ST.

1. ☐ **EQU:** Source A = `vfd_poll_step`, Source B = `2`.
2. ☐ **XIC:** `vfd_poll_active`.
3. ☐ **MSG_MODBUS** instance: `mb_write_cmd`. Pins: `LocalCfg = write_cmd_local_cfg`, `TargetCfg = write_cmd_target_cfg`, `LocalAddr = write_cmd_data`.

Config: `Channel := 2`, `Cmd := 6` (FC06 Write Single Register), `ElementCnt := 1`, `Addr := 8192` (0x2000), `Node := 1`.

Why this matters: Prog2 ST owns `vfd_cmd_word` and copies it into `write_cmd_data(1)` each scan. If you skip the copy, the write fires with stale data and the motor either won't start or runs the wrong way. v5.1 mentioned this as an optional MOV — make it mandatory.

Rung 3 — Write Frequency Setpoint

```
+---[EQU]-----[XIC]-----[MSG_MODBUS]-----+
| A = B          vfd_poll_active  mb_write_freq  |
| A = vfd_poll_step                                     |
| B = 3                                                  |
```

Writes one register to 0x2001 (Frequency Setpoint, value = Hz x 10).

1. ☐ **EQU:** Source A = `vfd_poll_step`, Source B = `3`.
2. ☐ **XIC:** `vfd_poll_active`.
3. ☐ **MSG_MODBUS** instance: `mb_write_freq`. Pins: `LocalCfg = write_freq_local_cfg`, `TargetCfg = write_freq_target_cfg`, `LocalAddr = write_freq_data`.

Config: `Channel := 2`, `Cmd := 6`, `ElementCnt := 1`, `Addr := 8193` (0x2001), `Node := 1`.

Frequency is Hz × 10, not Hz

To run at 30.0 Hz, load `300` into `vfd_freq_setpoint` — not `30`. Loading `30` sets 3.0 Hz, which is below most motor minimum frequencies and looks like the drive isn't responding. *[GS10 manual p.5]*

Rung 4 — Fault Reset

```
+---[EQU]-----[XIC]-----[MSG_MODBUS]-----+
| A = B          vfd_poll_active  mb_fault_reset  |
| A = vfd_poll_step                                     |
| B = 4                                                  |
```

Writes value `2` to register 0x2002 to clear any active fault on the VFD.

1. ☐ **EQU**: Source A = `vfd_poll_step`, Source B = `4`.
2. ☐ **XIC**: `vfd_poll_active`.
3. ☐ **MSG_MODBUS** instance: `mb_fault_reset`. Pins: `LocalCfg = fault_reset_local_cfg`,
`TargetCfg = fault_reset_target_cfg`, `LocalAddr = fault_reset_data`.

Config: `Channel := 2`, `Cmd := 6`, `ElementCnt := 1`, `Addr := 8194` (0x2002), `Node := 1`. Prog2 ST sets `fault_reset_cmd := 2` and copies it into `fault_reset_data(1)`.

7 Variable Map

Every variable that touches Prog3 — what it is, where it lives, and *why* it lives there. The "Why it belongs here" column is the actual teaching content. If you only memorize the names you'll fix this code once and be stuck the next time; if you understand the *why* you can design your own sequencer for a different drive.

Sequencer control (program-level, owned by Prog2)

Name	Type	Direction	Scope	Purpose	Why it belongs here
<code>vfd_poll_step</code>	INT	—	Global tag	Which rung is "in turn" right now (1..4).	Global because both Prog2 (advances it) and Prog3 (reads it on every rung's EQU) need to see the same value in the same scan. If it were local to one, the other couldn't react.
<code>vfd_poll_active</code>	BOOL	—	Global tag	Master enable — when FALSE, the sequencer is idle and no Modbus traffic happens.	Global so the safety interlock or HMI can pause polling in one place. Local to Prog2 would force you to thread it through to Prog3 as another parameter for no benefit.
<code>poll_timer</code>	TON	—	Local to Prog2	Times the ~50 ms step between rungs.	Local because no other program needs to see its <code>.ET</code> or <code>.Q</code> — Prog2 alone uses it to advance <code>vfd_poll_step</code> .

MSG_MODBUS instances (live in the Prog3 LD program)

Name	Type	Direction	Scope	Purpose	Why it belongs here
<code>mb_read_monitor</code>	MSG_MODBUS	—	Instance variable (Prog3)	The actual Modbus packet builder + state machine for the monitor read.	Each MSG_MODBUS instance has its own <code>.Done</code> , <code>.Error</code> , <code>.ErrorID</code> , plus internal scan-state. You need four separate instances because each polls a different register at a different time — sharing one would corrupt the wire.
<code>mb_write_cmd</code>	MSG_MODBUS	—	Instance variable (Prog3)	Same as above, for the command write.	See above.
<code>mb_write_freq</code>	MSG_MODBUS	—	Instance variable (Prog3)	Same as above, for the frequency-setpoint write.	See above.
<code>mb_fault_reset</code>	MSG_MODBUS	—	Instance variable (Prog3)	Same as above, for the fault-reset write.	See above.

Config structs (loaded each scan by Prog2 ST, consumed by Prog3 LD)

Name	Type	Direction	Scope	Purpose	Why it belongs here
<code>read_local_cfg / read_target_cfg</code>	struct	Input to MSG	Global tag	Tells <code>mb_read_monitor</code> which channel, function code, register, and node to talk to.	Global so Prog2 ST can write them; Prog3 LD wires them into the MSG block as inputs. Splitting into local + target is a Rockwell convention — Local = PLC-side settings, Target = drive-side settings.
<code>write_cmd_local_cfg / write_cmd_target_cfg</code>	struct	Input to MSG	Global tag	Same shape, for the command write.	Separate structs (not one shared) because the four polls have different Cmd values (3 vs 6) and different target Addrs. Sharing one struct would force you to mutate it mid-scan between rungs, which is exactly the kind of race the sequencer is designed to avoid.
<code>write_freq_local_cfg / write_freq_target_cfg</code>	struct	Input to MSG	Global tag	Same shape, for the frequency write.	Same.
<code>fault_reset_local_cfg / fault_reset_target_cfg</code>	struct	Input to MSG	Global tag	Same shape, for the fault reset write.	Same.

Data buffers (where MSG_MODBUS puts results / reads payloads)

Name	Type	Direction	Scope	Purpose	Why it belongs here
<code>read_data</code>	<code>INT[125]</code>	Output from MSG	Global tag	Receives the 4 monitor registers from rung 1. Indexing: <code>read_data(1)</code> = Output Frequency (Hz x10), <code>read_data(2)</code> = Output Current (A x10), <code>read_data(3)</code> = DC Bus Voltage (V), <code>read_data(4)</code> = Output Voltage (V). Indices 5+ are unused.	Global so Prog2 ST can scale and expose the values to other programs / HMI. If it were local to Prog3 you'd need a separate copy step.
<code>write_cmd_data</code>	<code>INT[125]</code>	Input to MSG	Global tag	Holds the command word for rung 2's write. Prog2 ST loads <code>write_cmd_data(1) := vfd_cmd_word</code> .	Global so Prog2 can load it. Buffer is sized 125 because that's the Rockwell convention for Modbus payload arrays even

Name	Type	Direction	Scope	Purpose	Why it belongs here
					though we only use index 1.
write_freq_data	INT[125]	Input to MSG	Global tag	Holds frequency setpoint × 10 for rung 3's write.	Same.
fault_reset_data	INT[125]	Input to MSG	Global tag	Holds 2 for rung 4's write (fault-reset bit).	Same.
vfd_cmd_word	INT	—	Global tag (set by HMI / state machine in Prog2)	The command Prog2 wants the VFD to obey right now (1 = stop, 18 = FWD+RUN, 20 = REV+RUN).	Global because the state machine that decides "we should be running forward now" lives elsewhere (Prog2). Prog3's only job is to ship the value out, not decide it.
vfd_freq_setpoint	INT	—	Global tag (set by HMI / state machine)	Desired motor frequency × 10. 300 = 30.0 Hz.	Same — separation of "decide what" (state machine) from "send it" (Prog3).

8 Code Walkthrough — scan by scan

Pick a moment when `vfd_poll_step = 2` and `vfd_poll_active = TRUE`. The state machine just decided we want REV+RUN, so `vfd_cmd_word = 20`. Walk what happens in one scan:

- 1. Input update phase.** The Micro 820 reads the physical input table — start button, e-stop, selector. None of those affect Prog3 directly; they affect `vfd_cmd_word` via the state machine in Prog2.
- 2. Prog2 ST executes.** Loads all four config structs (Channel=2, Cmd, Addr, Node). Loads `write_cmd_data(1) := vfd_cmd_word = 20`. Advances `poll_timer`; if `poll_timer.Q`, increments `vfd_poll_step` and resets the timer.
- 3. Prog3 LD executes.** Rung 1's EQU evaluates `vfd_poll_step = 1` → FALSE → rung dead. Rung 2's EQU evaluates `vfd_poll_step = 2` → TRUE. XIC sees `vfd_poll_active` → TRUE. `mb_write_cmd.IN` rises → the block builds an FC06 packet at 0x2000 with payload 20 and hands it to the serial driver. Rungs 3 and 4: their EQUs are FALSE; their MSGs sit idle.
- 4. Output update phase.** Nothing in Prog3 drives a physical output, so this is a no-op for our purposes.
- 5. Next scan.** Packet is still on the wire. `mb_write_cmd.Done` is FALSE because the drive hasn't replied yet. Rung 2 still satisfies EQU+XIC, but the MSG block won't re-fire because it's still in "waiting for

response" state internally.

6. **A few scans later** (~5–15 ms depending on baud and packet length), the drive replies.

`mb_write_cmd.Done` goes TRUE for one scan. `poll_timer` eventually completes; `vfd_poll_step` increments to 3. Rung 3 takes over.

Why this matters: The reason rung 2 doesn't re-fire while it's waiting for the drive is internal to `MSG_MODBUS` — the block has its own state machine. `EQU+XIC` being TRUE is necessary but not sufficient; the block also checks "am I currently in a transaction?" If yes, it ignores its `IN` until `.Done` or `.Error` fires. This is what protects the wire from packet stacking.

9 Scan Cycle Explanation

The Micro 820 runs a fixed five-phase loop every scan (default ~10 ms):

1. **Read inputs.** Physical input table copied into the internal image.
2. **Execute Prog2 (ST).** Loads configs, advances timer, increments `vfd_poll_step`.
3. **Execute Prog3-Modbus (LD).** Exactly one of the four rungs fires its MSG.
4. **Write outputs.** Output image copied to physical outputs.
5. **Housekeeping (comms, diagnostics).** The serial driver actually transmits the queued MSG packet during this phase — not during step 3. That's why MSG looks "instant" in the editor but actually takes effect at end-of-scan.

Knowing this matters because: if `poll_timer.PT = 50 ms` but the scan is 10 ms, `vfd_poll_step` advances every ~5 scans. Each MSG completes in ~2–3 scans (depends on packet length at 9600 baud). So the rotation comfortably fits within the timer window. If the scan ever stretches to 50+ ms (e.g., heavy logic added elsewhere), the timer fires every scan and MSG completions start colliding with step changes — you'll see `ErrorID` briefly bouncing through 55 (timeout) until the load drops.

10 Testing Procedure

1. ☐ Build the project: Ctrl+Shift+B. **0 errors.** If errors reference unknown variables, the CSV import didn't take — re-run it.
2. ☐ Download (USB cable, not Ethernet, for first download — Ethernet sometimes leaves the serial driver out of sync).
3. ☐ Switch PLC to Run mode.
4. ☐ Go online → Variable Monitor.

✓**You should see:** `vfd_poll_step` visibly cycles 1 → 2 → 3 → 4 → 1 at the timer rate. If stuck on one value, the timer in Prog2 is broken or `vfd_poll_active` is FALSE.

✓**You should see:** `mb_read_monitor.ErrorID = 0` (success) for at least 10 seconds. 255 means the MSG never executed (serial port out of sync — re-download). 55 means timeout (check wiring D+/D- swap, check P09.04=13).

✓**You should see:** `read_data(1) > 0` when motor is commanded to run. Value is Hz × 10, so 300 ≈ 30.0 Hz.

✓**You should see:** VFD keypad displays the commanded frequency (~30.0 Hz) within 2 seconds of `vfd_cmd_word := 18`.

Pass / Fail summary

Test	Pass	Fail	Notes
Project builds with 0 errors	☐	☐	
<code>vfd_poll_step</code> cycles 1→4	☐	☐	
<code>mb_read_monitor.ErrorID = 0</code>	☐	☐	
<code>read_data(1)</code> reads non-zero with motor running	☐	☐	
<code>vfd_cmd_word := 18</code> spins motor forward	☐	☐	
<code>vfd_cmd_word := 1</code> stops motor	☐	☐	
<code>vfd_cmd_word := 20</code> spins motor reverse (verify bit-math live)	☐	☐	If this fails, see open question §11 — try 18 vs 20 vs 34 on the bench.
Write 2 to <code>fault_reset_cmd</code> clears an active fault	☐	☐	

11 Troubleshooting Guide

Symptom	Likely cause	What to check	Fix
<code>mb_*.ErrorID = 255</code> on every rung	Serial driver not in sync (channel wrong, or download was partial)	Serial Port → Diagnose. Status should say "in sync".	Confirm <code>Channel := 2</code> in all four cfg structs. Re-download over USB. If still out of sync, try the factory-reset workflow from v5.1's Serial Port section.
<code>mb_*.ErrorID = 55</code> (timeout)	Wiring (D+/D- swap), baud mismatch, slave node wrong, or P09.04 wrong	VFD keypad: P09.00=1, P09.01=96, P09.04=13. Multimeter D+/D-: 2.0–2.5V DC idle bias.	Fix wiring or VFD params. Power-cycle the VFD after changing P09.xx.
Project builds but <code>mb_read_monitor</code> reports unknown identifier	You renamed the instance from v5.1's <code>mb_read_status</code> but didn't update Prog2 ST's references to <code>.Done / .ErrorID</code>	Search Prog2 for <code>mb_read_status</code> and rename to <code>mb_read_monitor</code> .	Find/Replace in Prog2.stf. Rebuild.
Frequency setpoint write succeeds but motor runs at wrong speed	Forgot the × 10 scaling	<code>vfd_freq_setpoint</code> value vs what VFD displays	Multiply your desired Hz by 10 before writing. 30 Hz → 300.
Direction command runs the wrong way	Bit-math contradiction between sources for FWD/REV command	Try the other value (18 ↔ 20 ↔ 34) on the bench and observe	Whichever value spins the motor forward is FWD+RUN for your specific GS10 firmware. Document the verified

Symptom	Likely cause	What to check	Fix
	word — see open question		value in the cheatsheet and update <code>GS10_Integration_Guide.md</code> .
Motor runs once then stops with CE10 / F30 on keypad	PLC scan stretched and poll timer collided with previous MSG completion	Add a <code>uptime_seconds</code> tag in Prog2; watch for scan-time spikes	Increase <code>poll_timer.PT</code> from 50 ms to 100 ms. If problem persists, reduce other program load.
Live monitor shows MSG block's logic but values look frozen	You're looking at the block definition , not the instance	Project Organizer → Prog3-Modbus → expand. Click <code>mb_read_monitor</code> (the instance), not <code>MSG_MODBUS</code> (the block type).	Open the instance's local variables view. Each MSG instance has its own state.
Fault won't clear even with rung 4 firing	Drive in F30.x latched state (comm-loss latch)	VFD keypad: STOP/RESET button. If still latched, power cycle.	Hardware reset; software reset is only enough for non-latched fault codes.

12 Mental Model

Why this matters: The polling sequencer is a rotating spotlight on a stage with four actors. Only the actor in the spotlight can speak. `vfd_poll_step` is the spotlight's current target. `poll_timer` is the stagehand who moves the spotlight on a 50 ms cadence. Each `MSG_MODBUS` instance is one actor, with one line to say into the serial wire. The wire is a stage with room for exactly one speaker — that's why the spotlight exists. Add a fifth actor someday? Just bump the modulo and add a fifth rung. The wire doesn't care; the spotlight handles the choreography.

13 What You Should Understand Now

- Why the four rungs all look the same and what changes between them (just the step number and the MSG instance).
- Why `Channel1 := 2` and not `0` for the Micro 820's embedded serial port — and why the deployed Prog2 ST is the authoritative source over the LD build guide when they disagree.
- That a `MSG_MODBUS` instance has its own internal "waiting for reply" state — the rung's EQU+XIC enables it, but the block decides when to actually send.
- Why each rung needs its own MSG instance and its own pair of cfg structs (sharing creates wire collisions and mid-scan struct mutation races).
- The difference between the status WORD (register 0x2101) and the live monitor BLOCK (0x2103+) — v5.1 conflated them and called the monitor read "status".
- Why the frequency value is multiplied by 10 (the GS10's frequency register is encoded as Hz × 10, integer).
- How the scan loop's five phases interact with MSG: the rung enables the block during phase 3, but the actual serial transmit happens in phase 5.

- What the timer cadence vs scan time relationship is and why a slow scan eventually corrupts the polling sequence.

14 Next Step

Two natural next guides, in order:

1. `Conv_Simple_Prog3_StatusWord_Decode.html` — add a **fifth** rung that reads register `0x2101` (the actual operation status word) and decode the bits (bit 1 = running, bit 2 = at-speed, bit 7 = fault active). This is what rung 1 was *supposed* to be doing in v5.1 per its instance name.
2. `Conv_Simple_UDFB_VFDCtrl.html` — wrap the cmd-write rung (Rung 2) into a CCW user-defined function block so you can re-use the same logic to drive two motors. Demonstrates the UDFB teaching arc against a topic you already understand mechanically.

15 Open question — bench-verify before relying on production

Command word bit math has a known contradiction between Mike's sources

Three different values for "REV + RUN" appear across the existing documentation:

- **20** (`0x0014`) — `plc/LD_BUILD_GUIDE_v5.md` Rung 2; `GS10_Integration_Guide.md` §2 "Common Command Words" table; deployed Prog2 ST.
- **18** (`0x0012`) — same sources, but labeled as FWD+RUN there.
- **34** (`0x0022`) — the existing `~/ .claude/skills/plc-instruction-guide/references/style-vocabulary.md` bug example says this is the correct REV+RUN value with explicit bit math "bit 5 + bit 1 = 32 + 2 = 34".

Per `GS10_Integration_Guide.md` §2's bit-field table: RUN sets bit 1 (decimal 2), FWD sets bit 3 (decimal 8), REV sets bit 4 (decimal 16). That arithmetic gives FWD+RUN = 10 and REV+RUN = 18. Neither matches the "Common Command Words" table on the same page, which lists FWD+RUN = 18 and REV+RUN = 20. The skill's bug example asserting 34 corresponds to a different bit-position convention again (bit 5 for REV).

What to do: bench-test all three values when you sit down at the rig. Whichever value *actually* spins the motor in reverse on your specific GS10 firmware is the correct one. Update this guide's cheatsheet and the integration guide with the bench-verified value and a [verified on bench YYYY-MM-DD] citation. Then delete the wrong values from the cheatsheet and from the v5.1 build guide so they don't propagate further. This is exactly the "when sources disagree" workflow in `~/ .claude/skills/plc-instruction-guide/references/citation-discipline.md`.

16 Appendix — cheatsheet repeat

① Tear-off cheat sheet (back-of-doc copy for quick reference)

What	Value
Channel (embedded RS-485 on Micro 820)	2
Baud / format	9600 8N2 RTU
Drive Modbus node	1
VFD RJ45 pin 5 = SG+ ; pin 4 = SG- ; pin 3 = SGND	—
FC03 monitor block	0x2103 (dec 8451), ElementCnt = 4 (NOT 6)
FC03 status word	0x2101 (dec 8449), ElementCnt = 1 (NOT in v5.1)
FC06 Control Command	0x2000 (dec 8192)
FC06 Frequency Setpoint	0x2001 (dec 8193), value = Hz × 10
FC06 Fault Reset	0x2002 (dec 8194), data = 2
Cmd: STOP	1
Cmd: FWD + RUN (deployed; bench-verify against bit-math, see §15)	18
Cmd: REV + RUN (deployed; bench-verify against bit-math, see §15)	20